

TODAY'S LEARNING

Classes and Objects

Day 5

Intermediate

10 min read

Generated by DevMentor AI by Dhruv Shukla

🔍 Today's Goal

After today's session, Dhruv will be able to design, instantiate, and manage C# classes and objects with an architecture-level understanding of heap allocation, object references, and lifecycle management. He will confidently model real-world business domains using rich encapsulated classes while avoiding memory bloat and reference bugs in ASP.NET Core applications.

🔍 Core Concept

What It Is & The Problem It Solves

A class is a reference type blueprint that defines state (fields/properties) and behavior (methods). An object is a concrete instance of that class allocated in memory.

Classes solve the problem of primitive obsession and scattered state by bundling data with the business rules that govern that data into a cohesive unit.

How It Works Internally

Memory Allocation: When you execute `new UserAccount()`, C# calculates the object's size (fields + 16-byte object overhead for `SyncBlockIndex` and `TypeHandle`).

Heap & Stack: The object's data is allocated on the Managed Heap. A reference pointer (4 or 8 bytes) pointing to this heap address is stored on the Stack or inside a containing object.

Initialization: Memory is zero-initialized, fields are assigned defaults, and the constructor (`.ctor`) executes to enforce invariant rules.

STACK

+-----+

MANAGED HEAP

+-----+

```
| accountRef (8 bytes)| -----> | TypeHandle & SyncBlockIndex |
+-----+          | Balance: 1500.00m          |
                  | AccountNumber: "ACC-9982"      |
+-----+          +-----+
```

Key Rules & Edge Cases

- Reference Equality vs. Value Equality: By default, `==` on class instances compares memory addresses, not the underlying property values.
- Nullability: Declaring a class reference without instantiation leaves it null. Accessing members on it throws a `NullReferenceException`.
- Parameterless Constructors: If no constructor is defined, the compiler supplies a default parameterless constructor. Defining any custom constructor removes this default.

What Happens If Done Wrong

Creating classes with public mutable fields leads to corrupted state across your application. Over-instantiating heavy classes inside high-frequency loops creates severe Garbage Collection (GC) pressure, causing latency spikes in production APIs.

Code Example

```
using System;

public class BankAccount
{
    // Encapsulated state
    public string AccountNumber { get; }
    public decimal Balance { get; private set; }

    // Constructor enforcing invariants
    public BankAccount(string accountNumber, decimal initialDeposit)
    {
        if (string.IsNullOrEmpty(accountNumber))
            throw new ArgumentException("Account number required.", nameof(accountNumber));
        if (initialDeposit < 0)
            throw new ArgumentOutOfRangeException(nameof(initialDeposit), "Initial deposit cannot be negative.");

        AccountNumber = accountNumber;
        Balance = initialDeposit;
    }

    public void Deposit(decimal amount)
    {
        if (amount <= 0) throw new ArgumentException("Deposit must be positive.", nameof(amount));
        Balance += amount;
    }
}
```

```
    }  
}  
  
public class Program  
{  
    public static void Main()  
    {  
        BankAccount account = new BankAccount("ACC-1092", 500.00m);  
        account.Deposit(250.00m);  
        Console.WriteLine($"Account {account.AccountNumber} Balance: ${account.Balance}");  
    }  
}
```

? Real Project Example

Business Scenario

In an e-commerce backend, we need an InventoryItem domain model that manages stock adjustments securely without allowing external services to directly overwrite stock quantities to invalid states.

```
using System;  
  
namespace Commerce.Domain  
{  
    public class InventoryItem  
    {  
        public Guid Id { get; }  
        public string Sku { get; }  
        public int AvailableQuantity { get; private set; }  
  
        public InventoryItem(Guid id, string sku, int initialStock)  
        {  
            Id = id == Guid.Empty ? Guid.NewGuid() : id;  
            Sku = !string.IsNullOrEmpty(sku) ? sku : throw new ArgumentException("Invalid SKU.");  
            if (initialStock < 0) throw new ArgumentException("Stock cannot be negative.");  
            AvailableQuantity = initialStock;  
        }  
  
        public void ReserveStock(int quantity)  
        {  
            if (quantity <= 0)  
                throw new ArgumentException("Quantity to reserve must be greater than zero.");  
            if (quantity > AvailableQuantity)  
                throw new InvalidOperationException($"Insufficient stock for SKU {Sku}.");  
        }  
    }  
}
```

```
        AvailableQuantity -= quantity;
    }
}
```

Explanation & Senior Architect Insights

Rich Domain Model: Properties have private set accessors. State modification is explicitly driven through business methods (ReserveStock).

Guaranteed Validity: The object cannot exist in an invalid state because the constructor enforces non-negative initial stock and valid SKUs.

Architect Note: Junior engineers often create Anemic Domain Models with public getters and setters for every property. A senior engineer enforces encapsulation so that invariants cannot be bypassed anywhere in the solution.

🔗 Top 3 Mistakes

1. Exposing Public Mutable Fields or Auto-Properties

Bad Code:

```
public class UserProfile
{
    public string Email; // Exposed public field
    public int Age { get; set; } // Open mutation without validation
}
```

Why it fails: Any caller can corrupt state (e.g., setting Age = -50 or Email = null), bypassing all domain rules.

Fix:

```
public class UserProfile
{
    public string Email { get; private set; }
    public int Age { get; private set; }

    public UserProfile(string email, int age)
    {
        UpdateEmail(email);
        SetAge(age);
    }
}
```

```
public void SetAge(int age)
{
    if (age < 0 || age > 120) throw new ArgumentOutOfRangeException(nameof(age));
    Age = age;
}

public void UpdateEmail(string email)
{
    if (string.IsNullOrEmpty(email) || !email.Contains("@"))
        throw new ArgumentException("Invalid email format.", nameof(email));
    Email = email;
}
}
```

2. Assuming Reference Comparison (==) Checks Object Content

Bad Code:

```
Customer c1 = new Customer("C100", "Dhruv");
Customer c2 = new Customer("C100", "Dhruv");

if (c1 == c2) // Evaluates to FALSE
{
    Console.WriteLine("Same customer");
}
```

Why it fails: The == operator on plain classes compares reference pointers on the stack, not data values on the heap. Two distinct allocations yield different memory addresses.

Fix: Override Equals and GetHashCode, implement IEquatable<T>, or use C# record types for value-based equality semantics.

```
public class Customer : IEquatable<Customer>
{
    public string Id { get; }

    public Customer(string id) => Id = id;

    public bool Equals(Customer? other) => other != null && Id == other.Id;
    public override bool Equals(object? obj) => Equals(obj as Customer);
    public override int GetHashCode() => Id.GetHashCode();
}
```

3. Allocating Heavy Objects Inside High-Frequency Loops

Bad Code:

```
for (int i = 0; i < 1_000_000; i++)
{
    var helper = new DataFormatter(); // Allocates 1M heap objects
    helper.Format(data[i]);
}
```

Why it fails: Rapid heap allocation triggers Gen 0 and Gen 1 Garbage Collection runs, stalling threads and creating latency spikes under load.

Fix: Instantiate the helper once outside the loop or use static methods if no instance state is required.

```
var helper = new DataFormatter();
for (int i = 0; i < 1_000_000; i++)
{
    helper.Format(data[i]);
}
```

Industry News

- Project Valhalla's First Preview: JEP 401 Redefines == for Java Objects

Java is introducing value objects via JEP 401 to eliminate identity overhead and redefine equality for non-identity types. Understanding object identity versus value identity across runtimes (such as C# structs/records vs classes) is essential for modern backend architects optimizing memory footprint.

 [Read full article](#)

- Using the GitHub Copilot SDK for Java

GitHub announced its SDK for programmatic integration of AI capabilities directly into application logic. As AI features move from IDE plugins into class-level SDK dependencies, developers must learn how to model AI context objects securely inside domain boundaries.

 [Read full article](#)

- Article: Comprehension as an Architectural Characteristic: A System That Is Not Understood Cannot Evolve Safely

This architectural deep-dive emphasizes that clear domain modeling and cohesive object design are crucial for long-term code maintainability. Overly complex object graphs and hidden state mutations severely compromise system comprehension and evolution.

[🔗 Read full article](#)

- CloudFlare Previews Automatic WebMCP Support for Web Pages

CloudFlare is integrating Model Context Protocol support into edge proxies, converting web interactions into machine-readable agent contexts. This shift highlights the growing demand for well-encapsulated object schemas that translate cleanly across client-server boundaries.

[🔗 Read full article](#)

- Canva Shares S3 Based Architecture for Session Revocation Across Hundreds of Millions of Sessions

Canva details how they manage session state objects distributed across global infrastructure. Designing scalable state objects with predictable lifecycles and cheap serializability is crucial when handling systems operating at immense scale.

[🔗 Read full article](#)

- How Pinterest Secures AWS Infrastructure at Scale with a Centralized Terraform Pipeline

Pinterest details infrastructure-as-code centralization to enforce security boundaries across cloud resources. Similar to class access modifiers in OOP, strong architectural isolation prevents unauthorized modifications across large engineering teams.

[🔗 Read full article](#)

- Java News Roundup: Shenandoah GC, TeamCity CVE, A2A Java SDK, Camel, Gradle, GlassFish, Groovy

This roundup highlights improvements in low-pause Garbage Collectors designed to collect unreferenced object instances faster. Understanding GC performance tuning is vital when building low-latency enterprise backends that instantiate high volumes of short-lived objects.

[🔗 Read full article](#)

[🔗 Interview Questions & Answers](#)

Q1: What is the difference between a class and an object in C#?

A1: A class is a reference-type blueprint defining properties, methods, and data layout. An object is a concrete instance of a class instantiated at runtime on the Managed Heap using the new keyword.

```
Car myCar = new Car(); // 'Car' is the class, 'myCar' references the created object
```

Q2: Where are class instances allocated in memory, and how are they referenced?

A2: Class instances (the object data, synchronization blocks, and type handles) are allocated on the Managed Heap. The variable pointing to that instance contains a memory address (reference pointer) stored on the Stack or within another heap object.

Q3: What happens during instantiation when the new keyword is executed?

A3: First, memory is allocated on the managed heap based on the type's size requirements. Second, memory is zero-initialized (default values assigned). Third, initializers run, and finally, the type's constructor (.ctor) is invoked to initialize fields and enforce rules.

Q4: What is the difference between Reference Equality and Value Equality for objects?

A4: Reference equality checks if two variables point to the exact same memory address on the heap. Value equality checks if two separate objects contain matching internal property values. Standard C# classes default to reference equality unless Equals or == are explicitly overridden.

```
var obj1 = new Person("Dhruv");  
var obj2 = new Person("Dhruv");  
bool refEqual = ReferenceEquals(obj1, obj2); // false
```

Q5: How does C# prevent an object from entering an invalid state upon creation?

A5: By using parameterized constructors, private setters, and validation guard clauses. By requiring valid data parameters during construction and avoiding parameterless constructors, an object ensures its internal state invariants are always met before any operations can occur.

```
public class Order  
{  
    public decimal Total { get; }  
    public Order(decimal total) => Total = total > 0 ? total : throw new ArgumentException();  
}
```

Q6: What is an Anemic Domain Model, and why is it considered an anti-pattern by architects?

A6: An Anemic Domain Model consists of classes with only auto-properties (get; set;) and no business logic or

invariants. It is an anti-pattern because domain logic gets leaked into external controllers or services, breaking encapsulation and allowing invalid object states to spread across the codebase.

Revision Summary

Day 4: Methods

- Key Idea: Methods encapsulate reusable execution logic and encapsulate business logic within classes.
- Remember: Use proper access modifiers (private, public, internal) and keep methods focused on a single responsibility (SRP) with clear return types and parameter validation.

Day 2: Operators

- Key Idea: Operators perform operations on operands (arithmetic, logical, relational, null-coalescing).
- Remember: Overloading == on custom reference types requires overriding Equals() and GetHashCode() to maintain consistent behaviors in hash collections like Dictionary<K,V>.

Tomorrow Preview

Tomorrow, we will explore Constructors and Initializers. You will learn how to write robust class initialization routines, chaining constructors with this, primary constructors in modern C# 12+, and enforcing strict object creation standards across enterprise microservices.